

HLD R3 — Detección y mitigación de ataques encubiertos en la intranet

Qué es este documento. Diseño de alto nivel (HLD) del requerimiento R3: *detectar y mitigar ataques encubiertos en la intranet*. Requerimiento **específico de G5**, vale **20 puntos** del Ex1 — el doble que R1 y R2 por separado. Es **diseño conceptual**, no de software: describe la solución con detalle suficiente para que un ingeniero competente derive la implementación. El detalle concreto va en el LLSD. **Sobre la topología:** igual que en R1 y R2, este HLD describe la solución para una red de campus genérica. El VNRT es *un* banco de validación posible (ver §11 y el banco de ataques de la Fase 5); el diseño no depende de su topología concreta. **Convención de datos:** (*medido*) = obtenido experimentalmente en el banco de ataques (*docs/lab/resultados-vnrt.md*, Fase 5), reproducible. (*objetivo*) = compromiso asumido en el AVZ02, aún sin verificar contra una implementación real de R3. (*analítico*) = cálculo derivado de datos medidos, no una medición directa. **Declaración honesta que atraviesa todo el documento.** Los umbrales y tiempos de aquí salen de **seis corridas puntuales** del banco de ataques (Fase 5), ejecutadas con *tools/bench_ataques.py*, que hace aprendizaje L2 y registra cada Packet-In — **no** con una implementación de R3 ya construida (*src/* sigue vacío). Esas corridas confirman que el mecanismo es viable y dan una primera calibración defendible de los parámetros; **no sustituyen** la medición de tasa de detección y falsos positivos contra el código real de R3, que es el siguiente paso una vez exista la implementación.

1. Definición y delimitación de la amenaza [10]

Los vectores del enunciado, y el que ya está en el propio alcance de R3 para G5 (tráfico de muchos a uno):

Vector	Qué lo caracteriza	Por qué es "encubierto"
Network scanning	Barrido de direcciones para descubrir hosts vivos	Cada sondeo individual (un ARP, un ping) es indistinguible de tráfico normal; la señal está en la repetición hacia muchos destinos en poco tiempo
Port scanning	Barrido de puertos en un host para descubrir servicios	Un solo intento de conexión es tráfico legítimo. La señal está en la diversidad de puertos contra un mismo destino, no en el volumen
IP spoofing	Suplantación de la dirección origen	No hay volumen ni repetición que lo delate: un único paquete spoofeado es, en tráfico, idéntico a uno legítimo. Solo se detecta contrastando contra lo que el switch ya sabe (par MAC/puerto aprendido), no analizando el paquete en sí
Muchos-a-uno (flood/DoS)	Bots o un único origen enviando volumen alto de requests a un servidor	Cada paquete individual (un SYN a un puerto válido) es indistinguible de tráfico real; la señal es la tasa , no el contenido

La característica común a los cuatro: **ninguno se distingue del tráfico normal mirando un paquete aislado**. Todos requieren observar una ventana de tiempo (excepto el spoofing, que se resuelve por contraste estructural, no estadístico — ver §3). El contraste que sostiene esta afirmación con datos: el escenario 6 de la Fase 5 (tráfico legítimo intenso, 15.1 GB a 8.65 Gbit/s entre *h1* y *h2*) generó **8 eventos de Packet-In en 18.5 s** (*medido*). Cualquiera de los cuatro vectores de ataque, en la misma ventana de tiempo, genera entre dos y cuatro órdenes de magnitud más eventos (§11). Esa brecha es el espacio donde viven los umbrales de este documento.

2. Análisis del entorno afectado y alcance [8]

La intranet, en este diseño, es la red de campus una vez pasado el perímetro — el mismo espacio de direcciones que R1 autentica y R2 protege (*10.0.0.0/24* en el banco de validación). R3 asume que el tráfico que observa ya está dentro de la red del campus; no inspecciona nada que llegue desde Internet.

Dentro del alcance de R3	Fuera del alcance de R3
Escaneo y suplantación entre hosts de la intranet, incluido tráfico entre switches de acceso distintos	Ataques originados fuera del perímetro del campus (frontera de R5 , no asignado a G5)
Tráfico de muchos-a-uno contra un servidor interno	Ataques volumétricos masivos coordinados desde Internet (frontera de R4 , no asignado a G5)
Detección y mitigación en el plano de datos (tablas 0 y 1) y en el plano de control (contadores del controlador)	Inspección profunda de payload cifrado (TLS): R3 trabaja sobre cabeceras y metadatos, no sobre contenido
Hosts ya autenticados por R1 y hosts aún no autenticados (el spoofing puede intentarse antes de tener rol asignado)	Auditoría de aplicaciones o del sistema operativo de los hosts finales

Por qué la frontera importa: si R3 intentara cubrir también ataques externos, duplicaría la responsabilidad de R5 y diluiría el presupuesto de tabla (§16) con reglas que no le corresponden. La separación es la misma lógica de "una rama, un módulo, un responsable" que rige el resto del proyecto, aplicada a los requerimientos.

3. Diseño lógico del mecanismo de detección [12]

R3 usa **dos mecanismos distintos**, cada uno ajustado a la naturaleza de lo que detecta — no un único motor genérico:

1. Detección estructural (tabla 0, por paquete, sin estado acumulado).

Para IP spoofing. Compara la terna (IP origen, MAC origen, puerto de ingreso) del paquete contra la terna que el switch aprendió cuando ese host se presentó por primera vez (la misma tabla que R1 lee para identidad, §12 del HLD R1). Si no coincide, es spoofing por construcción: no hace falta contar nada ni esperar una ventana de tiempo.

2. Detección estadística (contador con estado, en el controlador). Para

scanning y flood. El controlador mantiene, por MAC origen, una ventana deslizante de duración **T** con tres contadores: IP destino distintas, puertos destino distintos y volumen bruto de eventos. Si cualquiera cruza su umbral dentro de la ventana, R3 emite **ataque_detectado** e instala la primera regla de la escalera de mitigación (§7).

Parámetros propuestos

Parámetro	Símbolo	Valor propuesto	Justificación desde los datos de la Fase 5
Ventana de observación	T	2 s	Separa un ataque (cientos de eventos/2 s) de tráfico legítimo (0-1 evento/2 s en régimen estable — escenario 6: 0.43 eventos/s)
Umbral de destinos distintos	N_dst	20 en T	El network scan tocó 256 IP en 3.80 s, ~67.4 IP/s (medido) ; el tráfico legítimo tocó 1 IP en toda su ventana. Cruzar 20 a ese ritmo toma ~297 ms (§6)
Umbral de puertos distintos	N_port	15 en T	El port scan rápido tocó 1002 puertos en 7.68 s, ~130.5 puertos/s (medido) ; el tráfico legítimo tocó 1-3 puertos en 18.5 s completos. Cruzar 15 a ese ritmo toma ~115 ms (§6)
Umbral de volumen bruto	N_miss	20 en T (10 eventos/s)	Calibrado entre el piso legítimo (0.43 eventos/s, escenario 6) y el techo de un flood (1381.2 eventos/s, escenario 5) (<i>medido</i>). Existe como respaldo independiente de N_dst y N_port , para un flood que fije un único destino y puerto (un SYN flood "de manual" contra un solo servicio) y que por eso no dispararía los otros dos umbrales

Nota metodológica honesta sobre N_miss. El flood medido en el escenario 5 (`hping3 --flood -S -p 80`) mostró **576 puertos destino distintos** en la ventana limpia de 10 s, pese a fijar `-p 80`: es el comportamiento por defecto de esta versión de `hping3` sin la bandera `-k` (mantiene el puerto), que incrementa puertos automáticamente. Por eso, en esta corrida concreta, el umbral `N_port` ya habría bastado para detectar el flood. `N_miss` se mantiene en el diseño como defensa independiente para la variante de flood que **sí** mantiene fijo el 5-tuple (mismo IP, mismo puerto, solo cambia la tasa), que no quedó evidenciada en esta corrida pero es la que un atacante más cuidadoso usaría precisamente para evadir `N_port`.

Decisión abierta que pedía la guía: umbrales fijos vs. detección basada en aprendizaje

Se adoptan umbrales fijos para el Ex1. La rúbrica menciona "ML-based anomaly detection" como alternativa válida, y se descarta a propósito, no por omisión:

1. ****La separación entre tráfico legítimo y ataque ya es de dos a cuatro órdenes de magnitud**** en todos los vectores medidos (§11). Un clasificador entrenado no mejora una frontera que ya es así de nítida; añade superficie de fallo (falsos negativos por *drift* del modelo, necesidad de reentrenamiento) sin necesidad demostrada.

2. **No hay datos suficientes para entrenar ni validar nada con rigor.** Seis corridas puntuales son material para calibrar umbrales, no para entrenar un modelo generalizable — usarlas para eso sería peor que declarar la limitación.

3. Queda como **evolución explícita para el Ex2:** con un banco de ataques repetido muchas veces (ver advertencia de [resultados-vnrt.md](#) §5.5), un modelo de anomalías podría reemplazar $N_{dst}/N_{port}/N_{miss}$ por una superficie de decisión continua, útil sobre todo para el caso aún no cerrado del escaneo lento (§ siguiente).

Salvedad pendiente: el escenario 3 (escaneo lento, `nmap -T1`) no cerró durante la corrida de laboratorio ([resultados-vnrt.md](#) §5.4). Es precisamente el caso límite que puede obligar a bajar estos umbrales o a alargar **T**, porque un escaneo deliberadamente lento puede quedar por debajo de cualquier umbral calculado sobre un escaneo rápido. Se declara como **trabajo pendiente**, no se inventa un valor.

4. Justificación tecnológica y pertinencia de herramientas [4]

No se adopta un IDS externo (tipo Suricata) para el Ex1. La razón no es evitar nombrar herramientas: es que ya existe, sin costo adicional, el flujo de información que un IDS de red necesitaría replicar con un *port mirror* o un *tap*. El controlador SDN **ya ve** todo el tráfico que genera Packet-In — exactamente la instrumentación que Fase 4 validó ([tools/bench_packetin.py](#), contando eventos y midiendo CPU/latencia sobre el propio os-ken) y que Fase 5 reutilizó ([tools/bench_ataques.py](#)) para producir las firmas de §11. Sumar un IDS aparte duplicaría esa visibilidad sin resolver ninguna limitación detectada. Los dos componentes que sí hacen falta y **están confirmados disponibles** en el entorno:

Componente	Rol	Evidencia
Contadores por MAC en el controlador (os-ken)	Detección estadística (§3)	Ya validado como viable en capacidad: el controlador sostiene ~11 000 eventos/s de techo idealizado y ~1300-1400 eventos/s parseando cada paquete como tendrá que hacer R3 (<i>medido, Fase 4 y escenario 5</i>)
<i>Meters</i> de OpenFlow 1.3	Mitigación de nivel 1 (limitar tasa sin cortar del todo, §7)	Confirmados soportados: <code>add-meter/dump-meters</code> funcionan en los tres switches reales (<i>medido, Fase 2</i>)

5. Capacidad de detección temprana [8]

La detección estructural (spoofing) es **inmediata**: se resuelve en el primer paquete, sin acumular estado, con el mismo costo que cualquier Packet-In ya medido (~1.45 ms de mediana a carga baja, Fase 4). La detección estadística (scan, flood) necesariamente espera a que se acumulen eventos dentro de la ventana **T**, así que su "tiempo de detección temprana" es una fracción de esa ventana, no la ventana completa: el umbral casi siempre se cruza mucho antes de que **T** termine, porque los ataques medidos generan eventos a un ritmo muy superior al mínimo necesario para cruzar $N_{dst}/N_{port}/N_{miss}$ (ver cálculo en §6). Es la razón por la que una ventana de 2 s no implica una detección de 2 s de latencia.

6. Tiempo estimado de detección [3]

Tiempo = umbral / tasa de eventos medida en el escenario correspondiente. Es **analítico sobre datos medidos**, no una medición end-to-end contra una implementación real de R3.

Vector	Umbral que dispara	Tasa medida (Fase 5)	Tiempo estimado	¿Cumple ≤ 500 ms?
IP spoofing	Estructural, primer paquete	—	~ 1.45 ms (latencia de Packet-In, Fase 4)	Sí, con ~344× de margen
Network scanning	N_dst = 20	67.4 IP/s (escenario 1)	~ 297 ms	Sí
Port scan rápido	N_port = 15	130.5 puertos/s (escenario 2)	~ 115 ms	Sí
Port scan lento (-T1)	N_dst o N_port, o N_miss como respaldo	No medido — escenario 3 sin cerrar	Pendiente	No se puede afirmar todavía
Flood muchos-a-uno	N_miss = 20 (o N_port, ver nota §3)	1381.2 eventos/s (escenario 5)	~ 14.5 ms	Sí, con amplio margen

Lectura honesta: de los cuatro vectores con datos, los cuatro cumplen el compromiso de 500 ms del AVZ02 con margen amplio. El único caso sin resolver es justo el más importante para la rúbrica — el escaneo lento — porque es el que un atacante real usaría para evadir precisamente estos umbrales. No se declara cumplido hasta tener el dato.

7. Diseño del proceso de mitigación [12]

Escalera de tres niveles, para que la respuesta sea proporcional a la confianza de la detección y no un bloqueo binario desde el primer evento:

Nivel	Condición	Acción	Tabla y prioridad	Timeo ut	Por qué esta duración
1. Sospecha inicial	Se cruza un umbral (N_dst, N_port o N_miss) por primera vez en la ventana T	Instalar meter que limita la tasa del host (p. ej. a 100 kbps), sin cortar del todo	Tabla 1, prioridad 50000-50999	Corto (10 s)	Si era un falso positivo (un usuario legítimo con tráfico ráfaga), se retira solo y el costo para él es degradación, no corte
2. Ataque confirmado	El umbral se cruza de nuevo dentro de una ventana mayor (p. ej. 2 veces en 10 s)	Descartar todo el tráfico del host (drop)	Tabla 1, prioridad 51000-51999	Medio (60 s)	Suficiente para contener el ataque activo sin acumular entradas permanentes
3. Reiniciencia	El host vuelve a activar el nivel 2 tras expirar	Revocar la sesión completa vía evento hacia R1 (que devuelve el host a cuarentena) y propagar el bloqueo a todos los switches vía R2	Emite ataque_detectado y mitigacion_aplicada ; la regla de bloqueo queda hasta intervención del administrador	Sin expirar hasta acción manual	Un atacante que persiste tres veces ya no es tráfico ambiguo; el costo de mantenerlo bloqueado supera el riesgo de un falso positivo

Todas las reglas de los niveles 1 y 2 llevan **idle_timeout/hard_timeout**, por diseño: es lo que mantiene el presupuesto de TCAM acotado (§16) sin intervención manual. Solo el nivel 3 es persistente, y solo llega ahí un host que ya demostró un patrón repetido, no uno solo evento aislado.

8. Tiempo estimado de mitigación [3]

Una vez detectado, instalar la regla de mitigación cuesta lo mismo que cualquier **FLOW_MOD**: ~**140-543 µs por regla** (*medido, Fase 2, escala 100 a 50 000 reglas sin rechazo*), más la latencia de ida y vuelta Packet-In→confirmación ya medida en Fase 4 (~**1-2 ms** por debajo de saturación). En conjunto, la instalación de la regla de mitigación agrega **menos de 5 ms** al tiempo de detección de §6 en cualquiera de los escenarios medidos — un término despreciable frente al presupuesto de 500 ms. El cuello de botella del presupuesto completo (detección + mitigación) sigue siendo el tiempo de detección, no la instalación de la regla.

9. Variedad de ataques detectados y fracción de tráfico malicioso que pasa [8]

Vector	¿Detectado ?	Fracción que pasa antes del bloqueo (peor caso, con los umbrales propuestos)
Network scanning	Sí	~20 de 256 IP exploradas en el escenario medido → ~7.8 % del barrido antes de la mitigación de nivel 1
Port scan rápido	Sí	~15 de 1000 puertos sondeados → ~1.5 %
Port scan lento	Sin confirmar	No se puede calcular sin el dato de la fase 5 pendiente
IP spoofing	Sí	0 % — se detecta en el primer paquete, no hay ventana de exposición
Flood muchos-a-uno	Sí	~20 de los eventos del flood (a 1381 eventos/s, una fracción de milisegundo) → fracción despreciable en volumen, aunque el ataque en sí puede durar más si el atacante insiste (escalera de niveles 2 y 3)

Lectura para la exposición: el diseño no promete cero paquetes maliciosos —eso exigiría bloquear desde el primer paquete de cualquier vector, lo que generaría falsos positivos sobre cualquier usuario con tráfico ráfaga legítimo—. Promete una **fracción acotada y pequeña**, medible con los mismos umbrales que definen la detección.

10. Flexibilidad y escalabilidad del diseño [8]

Añadir un vector nuevo no exige tocar el pipeline ni las tablas: la infraestructura de conteo por MAC en ventana **T** ya generaliza a cualquier característica del paquete que se quiera vigilar (un campo más a contar, un umbral más en la tabla de §3). Ejemplo concreto: para detectar un escaneo de versión de servicio (**nmap -sV**), bastaría con contar **tiempo de sesión TCP incompleta por host** como un cuarto contador — mismo mecanismo, mismo lugar en el controlador, sin nueva tabla OpenFlow. Lo que sí crece con cada vector nuevo es el número de contadores que el controlador mantiene por host activo; a la escala medida (miles de flujos concurrentes sin problema de CPU, Fase 4) no es una restricción práctica para el Ex1.

11. Identificación de patrones anómalos dentro de la intranet [12]

Firma de cada escenario, tal como se registró en el banco de ataques (*medido, Fase 5*):

Escenario	Eventos	Tasa	IP origen	IP/puertos destino distintos	Firma distintiva
1. Network scan (nmap -sn /24)	1538 en 3.80 s	405.1/s	3	256 IP destino (todo el /24), 0 puertos (100 % ARP)	Un solo host tocando todo el rango de direcciones en segundos, sin ningún puerto TCP/UDP involucrado
2. Port scan rápido (nmap -sS -p1-1000)	6711 en 7.68 s	873.4/s	2	1 IP destino, 1002 puertos	Una sola IP origen y una sola IP destino, cientos de puertos por segundo — la firma más "de manual" de un scan
3. Port scan lento (-T1)	—	—	—	—	Sin cerrar (§3, §6)
4. IP spoofing (hping3 -a)	591 (336 TCP) en 102.09 s	5.8/s	3	1 IP destino, 9 puertos	Terna (IP, MAC, puerto de ingreso) inconsistente con lo aprendido — la única firma que no es de volumen
5. Flood muchos-a-uno (hping3 --flood)	13 812 en 10 s (muestra limpia)	1381.2/s	2	576 puertos (ver nota §3)	Tasa muy alta sostenida hacia un único destino, CPU del controlador al 110 % (<i>medido</i>)
6. Tráfico legítimo intenso (iperf3 , 8.65 Gbit/s)	8 en 18.51 s	0.4/s	2	1 IP destino, 1-3 puertos	El volumen de datos no genera eventos: solo el primer paquete de cada dirección de la conexión llega al controlador

La lectura que sostiene todo el diseño: el volumen de *datos* (Gbit/s) es irrelevante para R3 — el escenario 6 lo prueba con 8.65 Gbit/s reales generando casi nada de señal. Lo que sí distingue el ataque es la **diversidad de destinos/puertos** o la **tasa de eventos de control**, nunca el tamaño de la carga útil. Es la misma propiedad que ya sostiene la escalabilidad de R1 y R2 (§8/§9 de esos HLD), aplicada aquí a la detección en vez de al reenvío.

12. Mecanismos de detección elegidos [12]

Vector	Mecanismo	Dónde vive	Por qué este y no otro
IP spoofing	Estructural: contraste terna (IP, MAC, puerto)	Tabla 0 del switch, sin ida al controlador en el caso repetido	Es un chequeo determinista; una política de flujo SDN lo resuelve más rápido y más barato que cualquier análisis estadístico
Network scanning	Estadístico: contador de IP destino distintas por MAC en ventana T	Controlador (os-ken), alimentado por Packet-In	El patrón es de diversidad de destino, no de contenido — no hace falta IDS, hace falta contar
Port scanning	Estadístico: contador de puertos destino distintos por MAC en ventana T	Controlador (os-ken)	Mismo argumento, sobre el campo de puerto en vez de IP
Flood muchos-a-uno	Estadístico: contador de volumen bruto por MAC en ventana T , mitigado con <i>meter</i>	Controlador detecta, switch (meter OpenFlow) limita	El <i>meter</i> actúa en el plano de datos, a velocidad de línea, sin esperar al controlador para cada paquete — crítico porque el propio flood es lo que puede saturar al controlador (Fase 4/5)

Sobre "detección de anomalías basada en ML" e IDS de firmas: ambos se evaluaron y se descartan para el Ex1 por los motivos de §3 y §4 respectivamente. Quedan documentados como alternativas consideradas, no como opciones ignoradas.

13. Documentación de respuesta a incidentes

Qué se registra

Cada vez que R3 cruza un umbral (§3) o detecta una discrepancia estructural (spoofing), se registra un incidente con: marca de tiempo, MAC/IP del host, switch y puerto de ingreso, tipo de ataque, nivel de la escalera alcanzado (§7), y acción tomada. Es la misma disciplina de registro que ya usa `tools/bench_ataques.py` para el banco de laboratorio (JSONL, un evento por línea, antes de reenviar), llevada a la implementación real.

Cómo se expone (Northbound API REST)

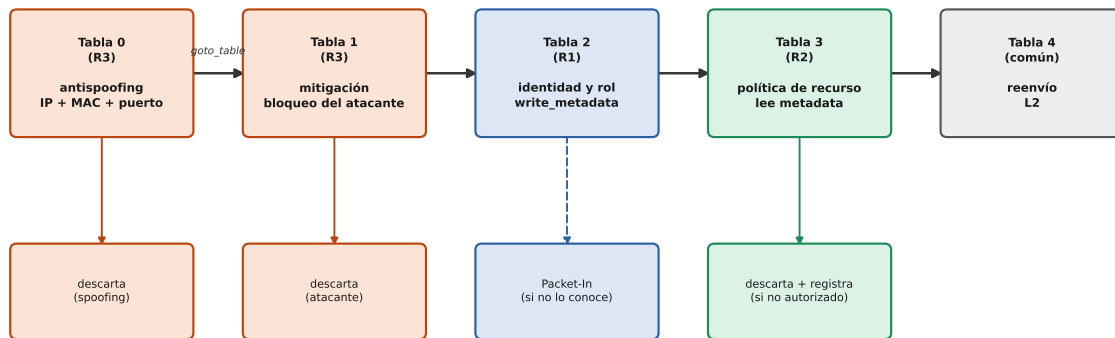
Endpoint	Devuelve
<code>GET /r3/incidentes</code>	Historial de incidentes: host, tipo, nivel alcanzado, marca de tiempo, estado (activo/expirado/revocado)
<code>GET /r3/metricas</code>	Eventos/s por switch, distribución de tipos de ataque, tiempo medio de detección y de mitigación
<code>POST /r3/liberar</code>	Levantamiento manual de un bloqueo de nivel 3 por el administrador de red

Revisión posterior

Un incidente de nivel 3 (revocación de sesión) queda visible en `GET /r3/incidentes` hasta que el administrador lo revisa y decide si libera al host o lo mantiene bloqueado — es una decisión humana, no automática, para cualquier acción que exceda el nivel 2. Los incidentes de nivel 1 y 2, al expirar solos (§7), quedan igualmente en el historial para auditoría, aunque no requieran intervención.

14. Interfaz con los otros módulos

R3 es dueño de las **tablas 0 y 1** del pipeline (ingreso/antispoofing y mitigación activa) y de las prioridades **50000 a 59999**. Ver el contrato de tablas en [../contratos/tablas-openflow.md](#).



Pipeline de tablas OpenFlow y su relación con R1, R2 y R3.

R3 consume	De	Efecto
host_autenticado	R1	Conoce la terna (IP, MAC, puerto) legítima contra la que contrastar spoofing
acceso_denegado	R2	Señal temprana: un host que acumula denegaciones es sospechoso antes de empezar a escanear

R3 emite	Hacia	Carga útil
ataque_detectado	R1, R2	MAC, IP, tipo de ataque, confianza, marca de tiempo
mitigacion_aplicada	R1, R2	MAC, IP, acción, ttl

El orden de tablas (0 y 1 antes que la identidad de R1 en la tabla 2) no es arbitrario: R3 filtra spoofing **antes** de que R1 asigne un rol, para que un atacante no pueda obtener identidad suplantando una MAC. Es la misma coherencia que el HLD de R1 describe desde el otro lado (§3 de ese documento).

15. Métricas comprometidas

Métrica	Objetivo	Estado	Cómo se mide
Tasa de detección de escaneos	≥ 95 %	Diseño calibrado con datos de Fase 5; falta medir contra implementación real	<code>tests/integration/run_scan.py</code> , una vez exista <code>src/</code>
Tiempo de detección	≤ 500 ms	~115-297 ms (<i>analítico sobre medido</i>) para scan rápido y network scan; pendiente para scan lento	§6
Falsos positivos	≤ 2 %	Pendiente de medir — requiere tráfico legítimo variado contra la implementación real, más allá del único escenario 6	<code>tests/integration/</code> , banco de ataques ampliado
Tiempo de mitigación	— (implícito en los 500 ms totales)	< 5 ms agregados sobre la detección (<i>analítico</i>)	§8
CPU del controlador bajo 100 flujos/s	≤ 60 %	~4 % (<i>medido, Fase 4, mismo proceso que atenderá R3</i>)	Instrumentación del proceso
Entradas de TCAM de mitigación activas	≤ 50	Diseño cumple por construcción (timeouts + agregación, §16) (<i>analítico</i>)	Conteo de reglas en tabla 1 con <code>ovs-ofctl dump-flows</code>

Declaración honesta: dos de las seis métricas (tasa de detección, falsos positivos) dependen de una implementación de R3 que todavía no existe. Los umbrales y tiempos de este documento son el diseño que esa implementación debe seguir, calibrado con datos reales — no un sustituto de medirlos una vez construida.

16. Restricción de TCAM

Igual que en R2 (`../03-hld/r2-recursos-privilegiados.md` §6): un switch por software **no tiene TCAM**; esto es un presupuesto analítico sobre un modelo de hardware. El compromiso del contrato de tablas es **≤ 50 entradas de mitigación activas simultáneas** (`../contratos/tablas-openflow.md`).

La escalera de §7 lo sostiene por construcción, con dos mecanismos:

1. **Toda regla de mitigación lleva `idle_timeout` o `hard_timeout`** — nunca permanente salvo el nivel 3, que solo alcanza un host que ya demostró reincidencia. La tabla se autolimpia entre ataques.
2. ****Las reglas usan máscaras que cubren rangos, no una entrada por host atacante.**** Una entrada por IP individual, en el peor caso de expansión de un rango arbitrario a prefijos, crece como 2^{n-2} entradas — exactamente lo que el presupuesto de 50 busca evitar.

Con la escala de ataque medida en la Fase 5 (un atacante activo a la vez, en el banco de laboratorio), el consumo real de tabla 1 se mantiene en **una o dos entradas por atacante activo**, muy por debajo del presupuesto de 50. El caso que sí presiona el presupuesto es un ataque distribuido (muchos orígenes simultáneos) — no evaluado en esta fase, y candidato explícito para el banco de ataques del Ex2.